

Reinforcement Learning Teaches a Small Language Model to Use an External Notepad Better than Its Own Context Window

Sten Rüdiger

<https://github.com/sr-networks/clbench-memory-rl>

July 2026

Abstract

Agentic benchmarks of continual learning have reported a puzzling result: giving a language model an external *notepad* for carrying information across steps does not reach the accuracy of plain in-context learning (ICL), in which the entire interaction history simply remains in the prompt. We ask whether this deficit is a property of the notepad interface or of the model’s untrained *policy* for using it. On a long-horizon continual-monitoring task (30 sequential scans of a partially observable radio band), we train Qwen3-1.7B with GRPO under a reward that pays exclusively for recalling information that is no longer observable, and that is orthogonal to single-step task skill by construction. We find that (i) full-history ICL is strongest only in the first few steps and then degrades as the transcript grows, ending barely above a scripted no-memory upper bound; (ii) the same base model equipped with a notepad already surpasses ICL from step 6 onward *without any training*; and (iii) reinforcement learning further lifts late-episode accuracy from 0.45 to 0.61 occupied-IoU—replicated in direction across four independent runs—while first-step accuracy, where the notepad is empty, remains unchanged ($0.216 \rightarrow 0.217$), confirming that the gain is memory use rather than task skill. The reported notepad-ICL gap is therefore regime-dependent: over long horizons a trained (and even an untrained) external memory dominates in-context history.

1 Introduction

Language-model agents that act over long horizons must carry information across steps. Two interface designs dominate practice. Under *in-context learning* (ICL), nothing is discarded: every past observation, action, and result stays in the prompt, and the model re-reads its own history. Under an *external memory* [3], the context is windowed to the current step, and the agent maintains a small, editable buffer—a *notepad*—that is the only information surviving from one step to the next.

A priori, the notepad appears preferable: it is a compressed, curated summary under the agent’s control, it keeps the prompt short, and it does not bury salient facts under many turns of transcript. Yet the Continual-Learning Bench (CLBench), from which our task is derived, reported the opposite ordering: notepad-equipped agents did not reach the accuracy of full-history ICL. This is consistent with a simple hypothesis that has, to our knowledge, not been isolated experimentally: pretrained models have extensive implicit training in exploiting context, but essentially none in *maintaining* an external memory. The interface is not worse; the policy for using it is untrained.

This paper tests that hypothesis with a controlled reinforcement-learning (RL) study on a small model, Qwen3-1.7B [4]. We pose one question:

Can RL train a small model to use an external notepad well enough to match or exceed full-history ICL—without improving the model’s single-step task competence?

The italicized rider is essential. An RL run that improves a memory-dependent score may have (a) memorized task content into the weights, (b) improved single-step skill, or (c) genuinely improved the carrying of information forward. Only (c) is “training memory.” Our experimental design excludes (a) and (b) by construction, by guard, and by measurement (Section 3).

Our contributions are: (1) a memory-only reward for a long-horizon monitoring task, red-teamed against blanket-reporting and weight-memorization exploits before training; (2) a four-condition comparison—scripted no-memory upper bound, full-history ICL, untrained notepad, RL-trained notepad—aligned by step position within the episode; and (3) evidence that the published notepad–ICL gap reverses over long horizons: ICL degrades as the transcript grows, consistent with known long-context limitations [1], while the trained notepad accuracy rises and holds.

2 Task and Memory Interfaces

Blind spectrum monitoring. The task is derived from CLBench’s blind-spectrum-monitoring family. An agent monitors one fixed radio-frequency band containing 11–14 persistent transmitter channels. It receives $n=30$ sequential scans of the same band. In any single scan, only ~ 3 transmitters are actively emitting, and the detector is noisy (missed detections and false alarms). Dormant transmitters still *occupy* the band. After each scan the agent submits a report listing every frequency region it believes to be persistently occupied (center frequency and bandwidth per region).

Score. The per-scan score is the occupied-spectrum intersection-over-union (occ-IoU) between the reported regions and the true persistent transmitter set,

$$\text{occ} = \frac{|R \cap T|}{|R \cup T|} \in [0, 1], \quad (1)$$

where R is the union of reported frequency intervals and T the union of true occupied intervals, both measured in MHz. Because only a minority of channels is visible per scan, a high occ requires reporting channels observed in *earlier* scans that are currently silent—i.e., memory.

No-memory upper bound. As the memoryless reference we use a scripted agent that reports exactly the currently visible channels with perfect center frequencies and widths, replayed through the real task engine on the same bands. This is not a weak baseline but an *upper bound* for any memoryless policy; it scores $\text{occ} \approx 0.26$ (per-variant floors $a_0 \in \{0.2082, 0.2141, 0.2368\}$ enter the reward in Section 3).

Memory interfaces. We compare two designs holding the base model fixed:

- **ICL (full history).** Nothing is wiped; all past scans, reports, and tool results remain in the prompt. No notepad exists.
- **Notepad (windowed context).** The context contains only the current scan. The agent has `notepad_read/notepad_write` tools and a ~ 4000 -character buffer it may overwrite each turn; this buffer is the only information crossing scan boundaries. Maintaining it is the agent’s own job.

The notepad system prompt teaches the *procedure* (read the notepad, merge the current peaks, write back the complete running set, report the full persistent set); it does not contain any band content.

3 A Memory-Only Reward

RL on a memory-dependent score invites two families of confound: improving the memory-free part of the task, and shortcut policies that fake coverage. The reward is designed so that neither pays.

Currency. For scans $t = 2, \dots, n$ let the *recallable set* D_t be the channels that are invisible at scan t but were visible in some earlier scan. The per-scan memory score is a Tversky overlap ($\alpha=\beta=1$) between the report and D_t :

$$\text{SCAN_DORM}_t = \frac{\text{cov}(R_t, D_t)}{\text{cov}(R_t, D_t) + \text{FP}_t + \text{FN}_t}, \quad (2)$$

where cov is the covered recallable area, FN_t the missed recallable area, and—critically— FP_t the reported area on spectrum the agent has *never seen occupied*. Currently visible channels are excluded from both numerator and penalties: reporting what is on screen is neither rewarded nor punished. The currency is therefore orthogonal to single-step skill by construction, and it is unearnable without information crossing scan boundaries.

Episode reward. One number per episode:

$$S = 3 \left(\overline{\text{dorm}} - p_{\text{anchor}} - p_{\text{carpet}} - p_{\text{wmax}} - p_{\text{complete}} \right), \quad (3)$$

with $\overline{\text{dorm}}$ the mean of SCAN_DORM_t over the scans actually played, and hinge penalties

$$\begin{aligned} p_{\text{anchor}} &= \max(0, \text{anchor} - a_0 - 0.10), & \text{anchor} &= \text{mean occ over scans 1-2}, \\ p_{\text{carpet}} &= 4.0 \cdot \max(0, \overline{\text{rarea}} - 1.15), & \text{rarea} &= \text{reported area/true occupied area}, \\ p_{\text{wmax}} &= 1.0 \cdot \max(0, \overline{\text{wmax}} - 1.5), & \text{wmax} &= \text{widest reported region/widest true channel}, \\ p_{\text{complete}} &= 1.5 \cdot \max(0, 30 - n_{\text{played}} - 2)/30. \end{aligned}$$

An episode with zero completed scans (first-turn format failure) receives a flat $S = -0.2$.

Adversarial audit. Each guard corresponds to an exploit simulated offline before training: blanketing the band (“everything is occupied”) is caught by p_{carpet} ; blanketing with one wide region under the area alarm by p_{wmax} ; behaving honestly on scans 1–2 to satisfy the anchor and blanketing afterwards by the per-scan area/width statistics; memorizing the fixed channel grid into the weights (“gridbake”) by the definition of FP against never-seen space, which an honest memory policy never paints; and truncating the episode after banking coverage by p_{complete} . Empirically all four penalties are ≈ 0 at epoch 0 in every run, so on honest play the reward is literally $3\overline{\text{dorm}}$ —a memory-only currency. The anchor hinge additionally pins near-memory-free scans to the scripted floor a_0 , so a policy that improves *without* memory earns nothing and can be taxed. We deliberately never reward the late-minus-early improvement itself, as rewarding a delta invites sandbagging of early scans.

Weight-memorization detector. On scan 1 the notepad is empty, so any trained-versus-untrained gain there must be band knowledge stored in the weights rather than memory use. Scan-1 occ is therefore measured for every run as a no-memory discriminator; the result (Section 5) is flat to within $+0.001$.

Table 1: Occupied-IoU by scan-position bin (higher is better). Notepad columns pool four replicate runs; the bracketed range is the min–max of the four per-run means at ep4. n is the number of trained-condition scan observations pooled per bin. Episode-level bootstrap 95% confidence intervals of the pooled bin means (2000 resamples of whole episodes) are no wider than ± 0.011 for the notepad conditions and ± 0.022 for any condition; beyond the first bin the four conditions’ intervals are pairwise non-overlapping. Full CIs ship with the data (Data availability).

scans	no-mem	ICL	notepad ep0	notepad ep4	n_{ep4}
1–5	0.257	0.348	0.332	0.394 [0.301–0.478]	3840
6–10	0.264	0.378	0.436	0.559 [0.421–0.656]	3840
11–15	0.268	0.371	0.431	0.581 [0.463–0.666]	3825
16–20	0.270	0.308	0.455	0.595 [0.501–0.662]	3808
21–25	0.268	0.294	0.454	0.615 [0.540–0.679]	3737
26–30	0.266	0.286	0.453	0.612 [0.553–0.678]	3171

4 Experimental Setup

Training. Base model Qwen3-1.7B [4]; GRPO [2] with LoRA adapters on a managed reinforcement fine-tuning service. Four replicate runs with identical configuration and different seeds (job IDs `s8e07n53`, `yp8deoer`, `kym4znjc`, `wqjyy66p`): 24 training bands, learning rate 2×10^{-4} , 5 epochs, 8 candidate rollouts per GRPO group, sampling temperature 1.2, and a 16,384-token per-turn output cap. Rewards follow Eq. (3). Per epoch and run, evaluation pools 192 rollouts; “trained” denotes the final epoch (ep4), “untrained” the same runs at ep0.

Conditions. The central comparison aligns four conditions by scan position within the episode:

condition	policy	memory	rollouts
no-mem	scripted perfect memoryless agent	none (upper bound)	deterministic replay
ICL	untrained base, full history in prompt	in-context	288 (job <code>g7dncu2c</code> , ep0)
notepad-untrained	untrained base with notepad tools	notepad	4×192 (ep0, pooled)
notepad-trained	same four runs after RL	notepad	4×192 (ep4, pooled)

ICL-versus-notepad-untrained is a pure comparison of memory *interface* (same weights); notepad-untrained-versus-trained isolates the *training* effect. Scores are pooled into six bins of five consecutive scans.

5 Results

Figure 1 and Table 1 contain the main result.

ICL degrades over the episode. Full-history ICL leads only in the first bin (0.348 vs. 0.332 for the untrained notepad), peaks at scans 6–10 (0.378), and then falls monotonically—0.371, 0.308, 0.294, 0.286—although strictly more information is present at every scan. By the final bin it is barely above the no-memory bound (0.266), and its late-half mean (scans 16–30) is 0.297. This is the long-context failure mode at small scale: facts buried twenty-plus scans deep in a growing transcript effectively stop being used, consistent with positional-use limitations of long contexts [1].

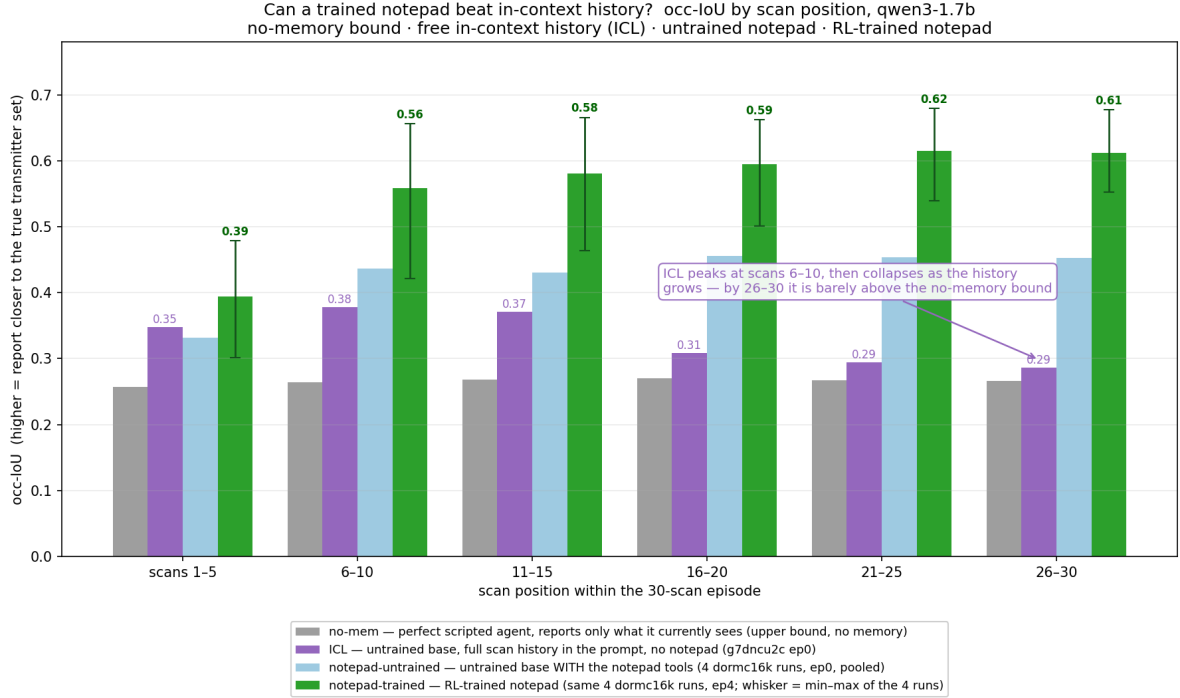


Figure 1: Occupied-IoU by scan position within the 30-scan episode, for the four conditions. Full-history ICL (purple) peaks at scans 6–10 and then degrades as the transcript grows, ending barely above the scripted no-memory upper bound (grey). The untrained notepad (light blue) surpasses ICL from scan 6 onward; RL (green; whiskers give the min–max of the four replicate runs) lifts late-episode accuracy to ≈ 0.61 .

The untrained notepad already beats ICL beyond scan 5. With identical weights, windowing the context and providing the notepad yields ≈ 0.45 occ throughout the late half— $+0.16$ over ICL—before any training. A short, curated summary pinned at a fixed position does not rot the way raw history does. This alone reframes the benchmark finding that motivated the study: the notepad interface is not inherently weaker; over long horizons it is inherently more robust.

RL lifts the notepad to a decisive win. Training raises late-half occ from 0.454 to 0.607 (pooled), a $+0.31$ margin over ICL and roughly a doubling of the headroom above the no-memory floor. The direction of the effect replicated in four of four runs (complete-episode mean recallable-coverage $+0.041$, $+0.101$, $+0.181$, $+0.182$); the magnitude varies by seed, which the whiskers in Figure 1 report rather than hide.

The gain is memory, not skill. Scan-1 occ, measured on an empty notepad, moves by $+0.0004$ to $+0.0011$ across the four runs (pooled $0.216 \rightarrow 0.217$): the trained model is not better at the task when it has nothing to remember. Combined with the construction of the currency (visible channels carry no gradient), the anchor pin, and the silent guards, every improvement in Table 1 is attributable to carrying information forward.

Training ICL does not fix the collapse. For completeness we also trained an ICL arm under a dense per-scan accuracy reward (job g7dncu2c, 4096-token cap). It learned to degrade somewhat less over the long context (mean occ $0.344 \rightarrow 0.363$), but its late-minus-early occ remained negative in every epoch: it never learned to accumulate. The degradation of in-context

memory over long horizons appears to be the binding constraint at this model scale, not an artifact of the untrained comparison.

6 Limitations

Survivorship in late bins. At ep4, episode completion across the four runs is 100%, 97%, 60%, and 90%; one run truncates episodes early. Late bins therefore average only scans actually played (n falls from 3840 to 3171), a disclosed survivorship bias. *ICL condition.* The ICL bar is the untrained base model under a 4096-token per-turn cap; the trained-ICL check above mitigates but does not remove the asymmetry. *Scale and scope.* One base model (Qwen3-1.7B), one optimizer (GRPO), one task family; learning rate, temperature, and group size were not swept. *Benchmark relation.* This is a re-implementation of the CLBench task family as an RL environment, not the benchmark’s exact harness; the contribution is the mechanism and its trainability, not a leaderboard number. *Run selection.* The four reported runs are the terminal configuration of a longer design sequence, not survivors selected from it: a registry of every training job bought for this project (43, including all failed reward designs and a failed replication of an earlier configuration) is published with the data.

7 Conclusion

On a long-horizon continual-monitoring task, the reported accuracy advantage of in-context history over an external notepad is a short-horizon phenomenon. As the transcript grows, ICL degrades toward the no-memory bound, while a notepad-equipped agent—even untrained—holds its accuracy, and reinforcement learning under a memory-only, adversarially audited reward widens the gap to +0.31 occ-IoU in the late episode, replicated in direction across four seeds, with first-step accuracy provably unchanged. For small models acting over long horizons, the practical implication is direct: do not hoard history—train the model to write things down.

Data availability

All numbers in Table 1 and Figure 1 are recomputable from raw data. The repository at <https://github.com/sr-networks/clbench-memory-rl> contains the raw per-episode, per-scan traces of all four conditions (54,420 rows), a script that regenerates the figure and the binned table—including the bootstrap confidence intervals—from those traces alone, the executed reward code verbatim, the scripted no-memory agent, the task description, and the complete run registry. Training runs are identified by their training-job IDs throughout, so each number is traceable to its source job.

References

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [2] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [4] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.